

আলু

- সময়সীমা: 10 মিনিট
- পরিবেশ: একটি GPU (≈16 GB VRAM), ইন্টারনেট নেই
- সমাধানের আকার: `solution.ipynb` ≤ 1 MB
- স্টোরেজ: 5 GB

কাজ

আপনার বন্ধু একটি অনুমানের খেলা খেলার প্রস্তাব দেয়। বিচারক হিসেবে সে একটি নির্দিষ্ট শব্দভাণ্ডার থেকে একটি গোপন শব্দ বেছে নেয়, এবং আপনাকে সর্বোচ্চ 30টি চালে সেটি খুঁজে বের করতে হবে। প্রতিটি চালে বিচারক দুটি শব্দের তুলনা করে এবং জানায় কোনটি গোপন শব্দটির অর্থগতভাবে বেশি কাছাকাছি। প্রতিটি খেলা নির্দিষ্ট জোড়া `lamp` vs `potato` থেকে শুরু হয়, কারণ এগুলো আপনার বন্ধুর সবচেয়ে প্রিয় জিনিসগুলোর মধ্যে দুটি। এরপর আপনার প্রোগ্রাম একটি নতুন শব্দ প্রস্তাব করে। তুলনায় বিজয়ী শব্দটি রেখে দেওয়া হয় এবং আপনার পরবর্তী প্রস্তাবের সঙ্গে তুলনা করা হয়। আপনি ঠিক গোপন শব্দটিই প্রস্তাব করার সঙ্গে সঙ্গে খেলাটি জিতে যান। মিল নির্ণয় case-insensitive। আপনার প্রস্তাবিত প্রতিটি শব্দ অবশ্যই `dataset/vocabulary.json`-এ থাকতে হবে।

প্রোটোকল ও ডেটা লোডিংসহ একটি পূর্ণাঙ্গ উদাহরণ `solution.ipynb`-এ রয়েছে। আপনি `PublicEmbeddingPlayer` class পরিবর্তন করতে পারেন। আপনার প্রোগ্রাম একবার `initialize` করা হয় এবং একটি একক `run`-এ প্রতিটি খেলা খেলে; প্রোটোকল প্রতিটি খেলার শুরুতে একটি নতুন `PublicEmbeddingPlayer` তৈরি করে।

বিচারক

আপনার প্রোগ্রাম বিচারকের কাছে একটি JSON object পাঠায় এবং বিচারক একটি JSON object দিয়ে উত্তর দেয়।

শুধু প্রোটোকলটি ব্যাখ্যা করার জন্য গোপন শব্দ দেখানো একটি পূর্ণাঙ্গ উদাহরণ:

```
Hidden word: shovel          Fixed opening: lamp vs potato

<- {"turn": 1, "winner_word": "potato", "verdict": "second", "word1": "lamp", "word2": "potato"}
-> {"new_word": "rock"}
<- {"turn": 2, "winner_word": "rock", "verdict": "second", "word1": "potato", "word2": "rock"}
-> {"new_word": "hammer"}
<- {"turn": 3, "winner_word": "hammer", "verdict": "second", "word1": "rock", "word2": "hammer"}
-> {"new_word": "shovel"}          <- matches: game won
-> {"status": "win"}
```

চালগুলো 1 থেকে 30 পর্যন্ত index করা হয়।

`verdict`-এর সম্ভাব্য মান হলো `first`, যার অর্থ `word1` বেশি কাছাকাছি; `second`, যার অর্থ `word2` বেশি কাছাকাছি; অথবা `same`, যার অর্থ উভয় শব্দই গোপন শব্দটির সমান কাছাকাছি।

`winner_word` হলো পরবর্তী তুলনার জন্য রেখে দেওয়া শব্দ। `same` রায়ের ক্ষেত্রে প্রথম শব্দটিই থেকে যায়।

Dataset

প্রতিটি `split`-এর জন্য অভিন্ন:

- `dataset/vocabulary.json` — 1602টি স্বতন্ত্র lowercase শব্দ। গোপন শব্দটি সর্বদা এগুলোর একটি।
- `dataset/public_embeddings.npy` — `float32`, `shape` (1602, 2560)। Row `i` শব্দভাণ্ডারের `i` শব্দটির সঙ্গে সম্পর্কিত। এগুলো `public` embedding; বিচারক একটি ভিন্ন, `private representation` ব্যবহার করে।

`split`-গুলো গোপন শব্দের `set`:

Split	শব্দ	উত্তর	যে কাজে ব্যবহার করবেন
test_public	120	✅ <code>dataset/test_public.json</code>	আপনার সমাধান run করতে এবং নিজে score করতে
test_leaderboard_a	120	গোপন	live leaderboard
test_leaderboard_b	120	গোপন	চূড়ান্ত ranking

কোনো `train split` নেই — labelled row থেকে কিছুই fit করা হয় না।

প্রদত্ত model

Task-এর সঙ্গে দুটি pretrained embedding model দেওয়া আছে এবং সেগুলো ব্যবহার করা যেতে পারে:

```
models/Qwen/Qwen3-Embedding-0.6B
[Qwen Embedding model card] (https://yastatic.net/s3/contest/ioai/3/01_Qwen_Qwen3-Embedding-0.6B_MODEL_CARD.html)
models/BAAI/bge-m3
[bge-m3 model card] (https://yastatic.net/s3/contest/ioai/3/02_BAAI_bge-m3_MODEL_CARD.html)
```

উভয়টিকেই তাদের local path থেকে load করতে হবে; "BAAI/bge-m3"-এর মতো একটি Hugging Face hub id download শুরু করে এবং ব্যর্থ হয়, কারণ judging offline। প্রতিটি directory-তে offline call দেখানো একটি চালানো যায় এমন example.py রয়েছে।

উপলভ্য library: numpy, torch, sentence-transformers। ইন্টারনেট নেই, download নেই, অন্য কোনো package নেই।

Output

কিছুই নয়। এটি একটি interactive task: আপনার সমাধান কোনো answer file লেখে না; এটি উপরে বর্ণিতভাবে stdin/stdout-এর মাধ্যমে বিচারকের সঙ্গে যোগাযোগ করে।

Metric

t নম্বর চালে খুঁজে পাওয়া একটি খেলার score $1.0 - 0.02 \times \max(0, t - 10)$; 30টি চালের মধ্যে সমাধান না হওয়া একটি খেলার score 0। অতএব 1-10 নম্বর চালের score 1.00, 20 নম্বর চালের score 0.80 এবং 30 নম্বর চালের score 0.60।

আপনার task score হলো game score-এর গড় $\times 100$, যা 0.00 থেকে 100.00-এর মধ্যে।

10-মিনিটের সীমাটি start-up, প্রস্তুতি এবং test set-এর সব 120টি খেলার জন্য একটি একক budget।

যেভাবে submit করবেন

1. solution.ipynb খুলুন, PublicEmbeddingPlayer edit করুন এবং এটি কাজ করছে কি না নিশ্চিত করতে সব cell run করুন।
2. ঐচ্ছিকভাবে, local-এ এটি পরীক্ষা করুন: `python local_test.py solution.ipynb --limit 5`। local বিচারক *public* embedding ব্যবহার করে, তাই এর score কেবল একটি নির্দেশক।
3. solution.ipynb save করুন।
4. JupyterLab-এর বাম sidebar-এ Git tab খুলুন।
5. solution.ipynb stage করুন (এর পাশের + icon)।
6. একটি commit message লিখে Commit-এ click করুন।
7. push করতে উপরের দিকে তিরচিহ্নসহ cloud icon-এ click করুন।
8. এই Contest page-এ ফিরে এসে Submit-এ click করুন; commit message-টি আপনার দেওয়া message-এর সঙ্গে মিলতে হবে।

প্রয়োজনীয় যেকোনো প্রস্তুতি ও inference-সহ ঠিক একটি file submit করুন, যার নাম solution.ipynb।